

TECHNICAL REPORT

MODULE 1 — SELENIUM AUTOMATION

Scope

Six completed Selenium/Python practical assignments covering locators, explicit waits, dynamic controls, JavaScript alerts, HTML tables, iframes, and browser windows.

No.	Assignment
1	The Multi-Locator Challenge
2	Dynamic Content & Explicit Waits
3	Dynamic Dropdowns & Checkboxes
4	JavaScript Alerts and Confirms
5	HTML Web Table Extractor
6	Windows, Tabs, and Iframes

The execution screenshots in this report are reproduced in their original full-screen form.

The Multi-Locator Challenge

Description

Navigate to a login page and use different Selenium locator strategies for the username, password, and login button. Validate that the resulting URL contains /inventory.html.

Python / Selenium Implementation

```
from selenium import webdriver
from selenium.webdriver.common.by import By

# Open Chrome
driver = webdriver.Chrome()

# Open SauceDemo login page
driver.get("https://www.saucedemo.com/")

# Enter username using By.ID
username = driver.find_element(By.ID, "user-name")
username.send_keys("standard_user")

# Enter password using By.NAME
password = driver.find_element(By.NAME, "password")
password.send_keys("secret_sauce")

# Click login button using By.XPATH
login_button = driver.find_element(
    By.XPATH,
    "//input[@type='submit']"
)
login_button.click()

# Validate successful login
assert "/inventory.html" in driver.current_url

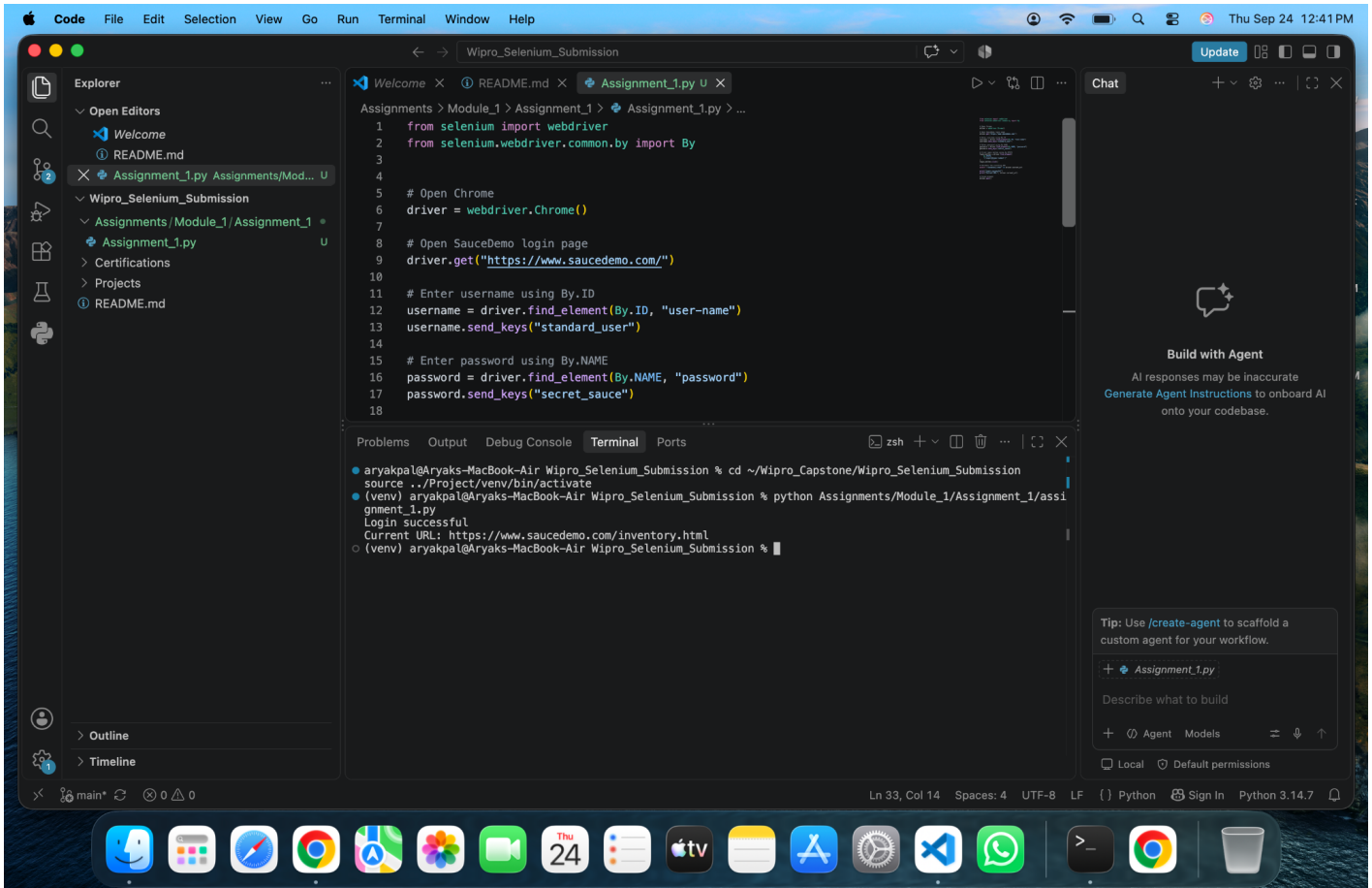
print("Login successful")
print("Current URL:", driver.current_url)

driver.quit()
```

Execution evidence is presented on the following page using the original full-screen screenshot.

Assignment 1 — Execution Evidence

Original screenshot supplied from the completed execution. The image is preserved intact; only proportional scaling is applied to fit the page.



Description

Open a dynamic-loading page, trigger the loading action, wait explicitly for the generated message, and validate that the displayed text is Hello World!.

Python / Selenium Implementation

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 10)

driver.get(
    "https://the-internet.herokuapp.com/dynamic_loading/1"
)

# Click Start
driver.find_element(
    By.XPATH,
    "//button[text()='Start']"
).click()

# Wait for dynamic content
message = wait.until(
    EC.visibility_of_element_located(
        (By.CSS_SELECTOR, "#finish h4")
    )
)

# Extract the text
text = message.text

# Validate the text
assert text == "Hello World!"

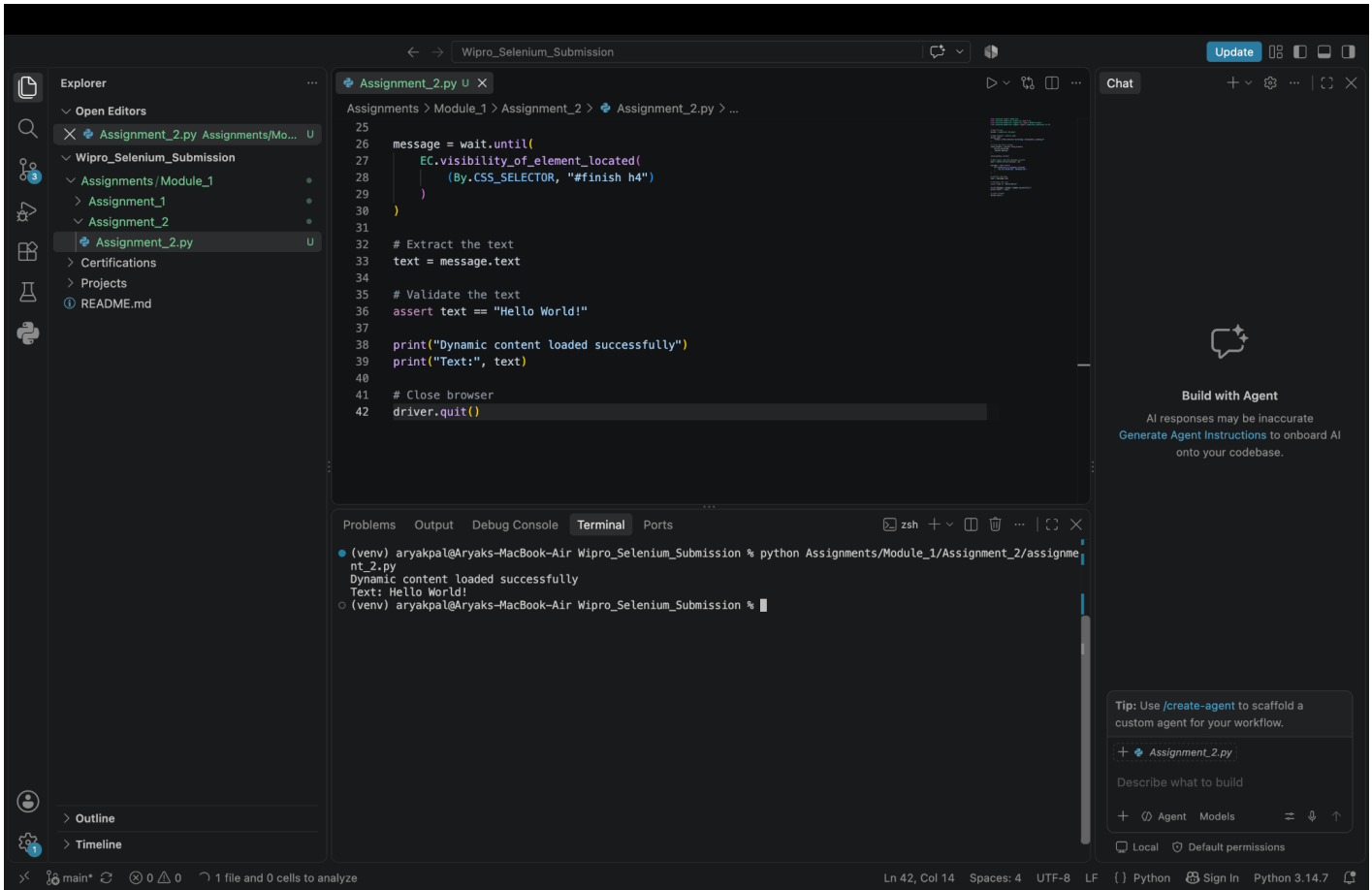
print("Dynamic content loaded successfully")
print("Text:", text)

driver.quit()
```

Execution evidence is presented on the following page using the original full-screen screenshot.

Assignment 2 — Execution Evidence

Original screenshot supplied from the completed execution. The image is preserved intact; only proportional scaling is applied to fit the page.



Description

Select specific checkboxes and verify their state using `is_selected()`. Then type into an auto-suggest field, iterate through the suggested results, and select India.

Python / Selenium Implementation

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 10)

driver.get("https://rahulshettyacademy.com/AutomationPractice/")

checkbox1 = wait.until(
    EC.element_to_be_clickable(
        (By.ID, "checkBoxOption2")
    )
)

checkbox2 = wait.until(
    EC.element_to_be_clickable(
        (By.ID, "checkBoxOption3")
    )
)

if not checkbox1.is_selected():
    checkbox1.click()

if not checkbox2.is_selected():
    checkbox2.click()

assert checkbox1.is_selected()
assert checkbox2.is_selected()

print("Checkboxes selected successfully")

driver.get("https://rahulshettyacademy.com/dropdownsPractise/")

search_box = wait.until(
    EC.visibility_of_element_located(
        (By.ID, "autosuggest")
    )
)

search_box.send_keys("ind")

suggestions = wait.until(
    EC.visibility_of_all_elements_located(
        (By.CSS_SELECTOR, "li[class='ui-menu-item'] a")
    )
)

for suggestion in suggestions:
    if suggestion.text == "India":
        suggestion.click()
        break

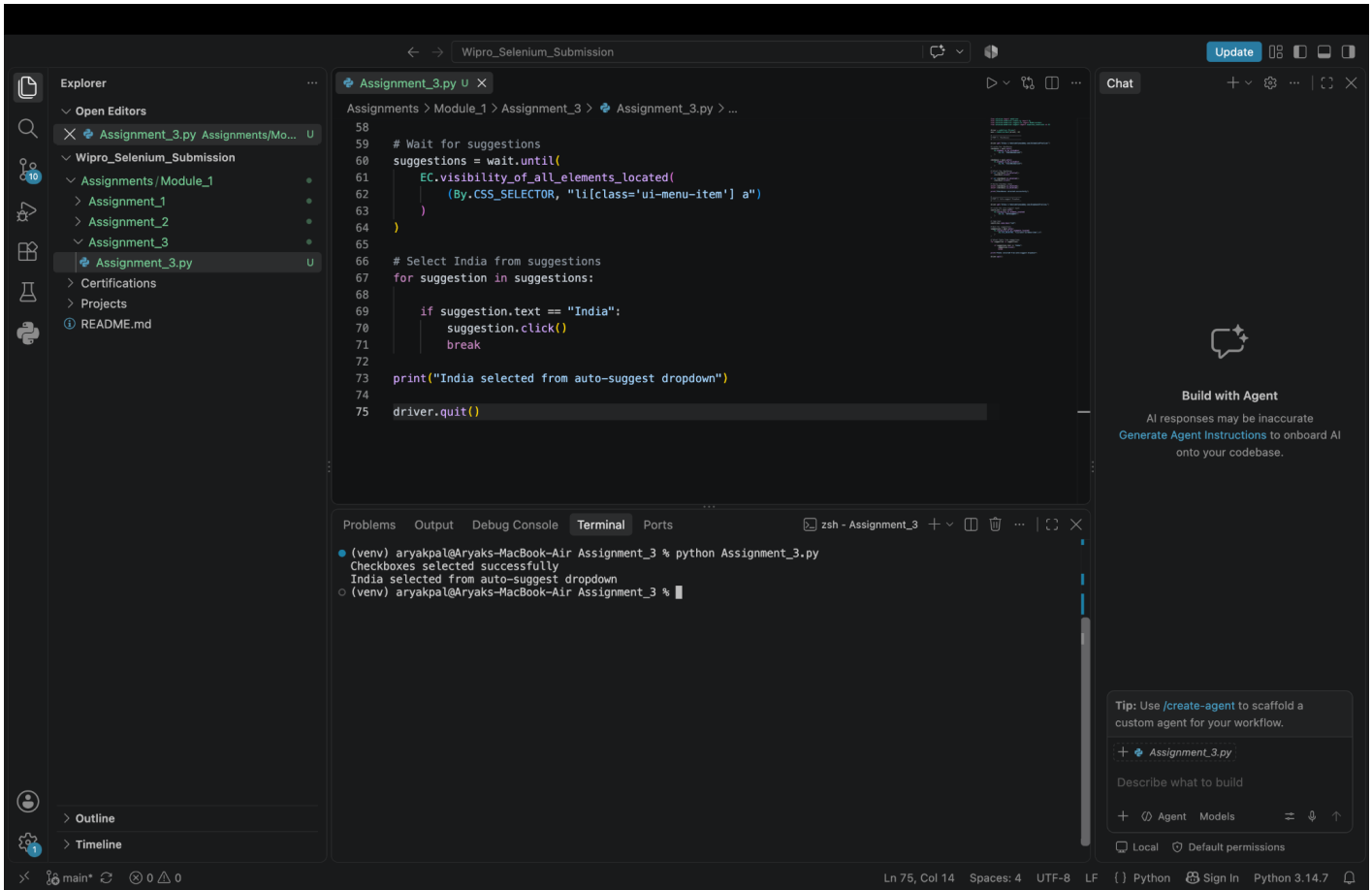
print("India selected from auto-suggest dropdown")

driver.quit()
```

Execution evidence is presented on the following page using the original full-screen screenshot.

Assignment 3 — Execution Evidence

Original screenshot supplied from the completed execution. The image is preserved intact; only proportional scaling is applied to fit the page.



Description

Trigger a JavaScript Alert, a Confirm Box, and a Prompt Box. Accept the first alert, dismiss the confirm box, and enter text into the prompt before accepting it.

Python / Selenium Implementation

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 10)

driver.get("https://the-internet.herokuapp.com/javascript_alerts")

driver.find_element(
    By.XPATH,
    "//button[text()='Click for JS Alert']"
).click()

alert = wait.until(
    EC.alert_is_present()
)

alert.accept()

print("JavaScript Alert accepted successfully")

driver.find_element(
    By.XPATH,
    "//button[text()='Click for JS Confirm']"
).click()

alert = wait.until(
    EC.alert_is_present()
)

alert.dismiss()

print("JavaScript Confirm dismissed successfully")

driver.find_element(
    By.XPATH,
    "//button[text()='Click for JS Prompt']"
).click()

alert = wait.until(
    EC.alert_is_present()
)

alert.send_keys("Hello Selenium")

alert.accept()

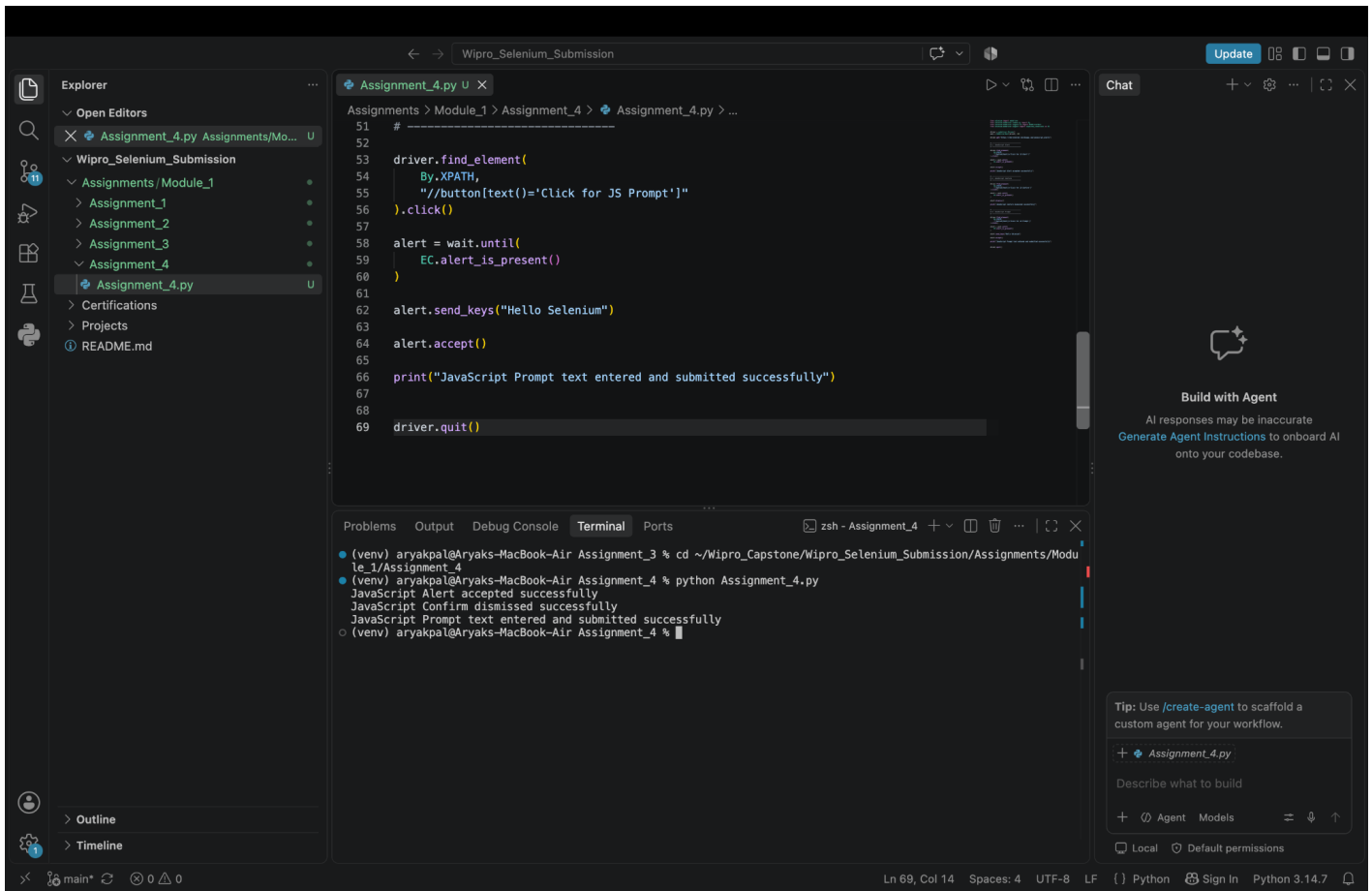
print("JavaScript Prompt text entered and submitted successfully")

driver.quit()
```

Execution evidence is presented on the following page using the original full-screen screenshot.

Assignment 4 — Execution Evidence

Original screenshot supplied from the completed execution. The image is preserved intact; only proportional scaling is applied to fit the page.



Description

Find a multi-column HTML data table, iterate through its rows and columns, locate the row matching Smartphone, and retrieve the corresponding price.

Python / Selenium Implementation

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 10)

driver.get("https://testautomationpractice.blogspot.com/")

table = wait.until(
    EC.visibility_of_element_located(
        (By.ID, "productTable")
    )
)

rows = table.find_elements(
    By.TAG_NAME,
    "tr"
)

search_name = "Smartphone"

for row in rows:
    columns = row.find_elements(
        By.TAG_NAME,
        "td"
    )

    if len(columns) > 0:
        product_name = columns[1].text
        price = columns[2].text

        if product_name == search_name:
            print("Product found:", product_name)
            print("Price:", price)

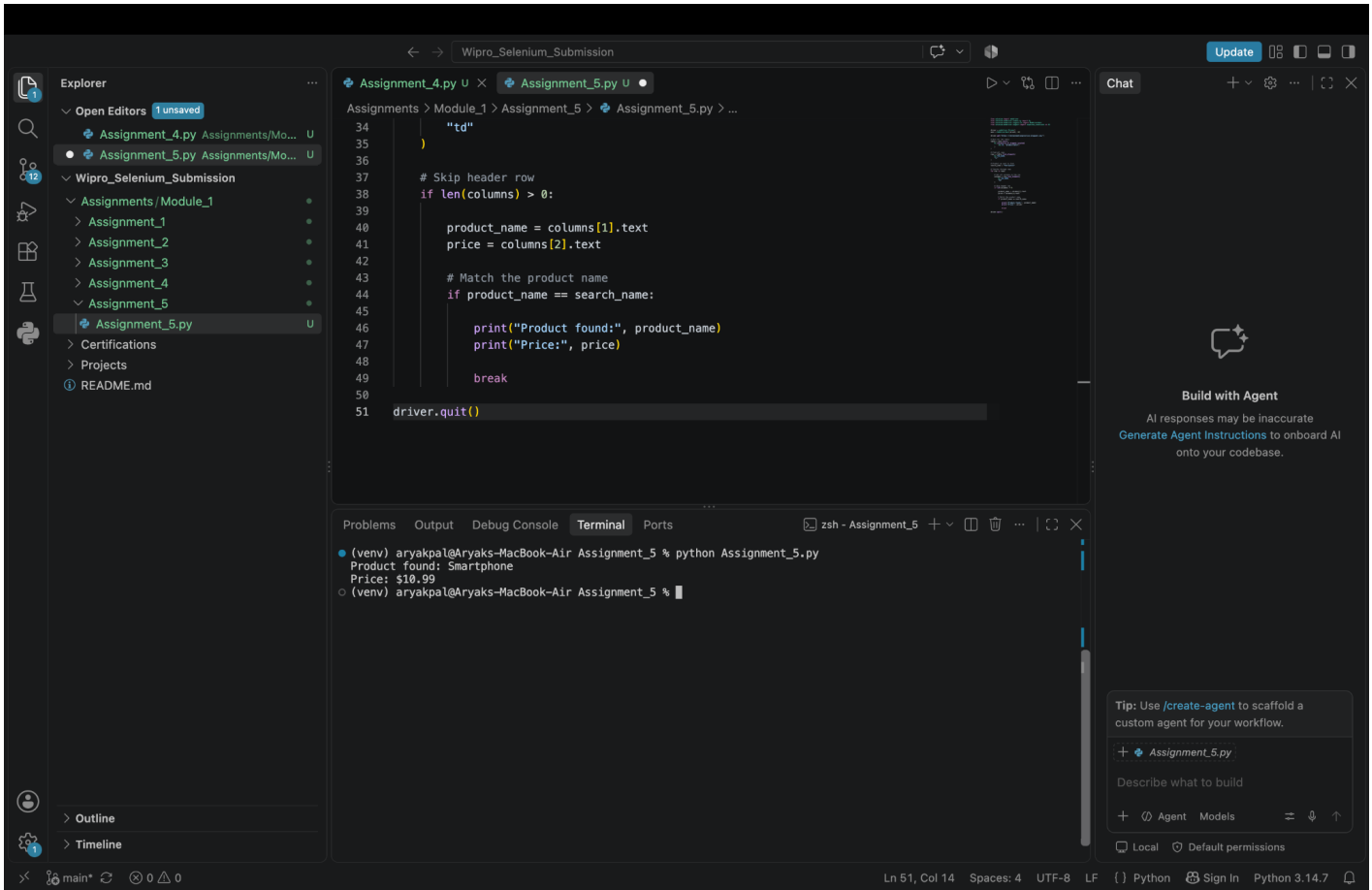
            break

driver.quit()
```

Execution evidence is presented on the following page using the original full-screen screenshot.

Assignment 5 — Execution Evidence

Original screenshot supplied from the completed execution. The image is preserved intact; only proportional scaling is applied to fit the page.



Description

Use `driver.switch_to.frame()` to interact with an iframe, then use window handles to switch to a new browser window, retrieve its title, close it, and return to the main window.

Python / Selenium Implementation

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 10)

# -----
# PART 1: Handle IFrame
# -----

driver.get("https://the-internet.herokuapp.com/iframe")

# Find the iframe
iframe = wait.until(
    EC.presence_of_element_located(
        (By.ID, "mce_0_ifr")
    )
)

# Switch to the iframe
driver.switch_to.frame(iframe)

# Find the text editor inside the iframe
editor = wait.until(
    EC.visibility_of_element_located(
        (By.ID, "tinymce")
    )
)

# Select existing text and enter new text
editor.send_keys(Keys.COMMAND, "a")
editor.send_keys("Hello from Selenium")

print("Text entered inside iframe successfully")

# Switch back to the main page
driver.switch_to.default_content()

print("Switched back to main page")

# -----
# PART 2: Handle New Tab
# -----

driver.get("https://the-internet.herokuapp.com/windows")

# Store the main window
main_window = driver.current_window_handle

# Click the link that opens a new tab
driver.find_element(
    By.LINK_TEXT,
    "Click Here"
).click()

# Get all open windows
windows = driver.window_handles

# Switch to the new tab
for window in windows:
    if window != main_window:
        driver.switch_to.window(window)
        break

# Get the title of the new tab
print("New tab title:", driver.title)

# Close the new tab
driver.close()

# Switch back to the main window
driver.switch_to.window(main_window)

print("Switched back to main window successfully")

# Close the browser
driver.quit()
```

Execution evidence is presented on the following page using the original full-screen screenshot.

Assignment 6 — Execution Evidence

Original screenshot supplied from the completed execution. The image is preserved intact; only proportional scaling is applied to fit the page.

